Programming with Python
IDC 212

July 27, 2025

Contents

1. Introduction

1.1 History
1.2 Application
1.3 Features
1.4 Comparison
1.5 Interpreted Language
1.6 Python Vs C++
1.7 Installation
1.8 Writing First Program
1.9 Python Syntax and Features

Introduction

Introduction

- Python is a general purpose high level programming language.
- Python was developed by Guido Van Rossum in 1989 while working at the National Research Institute in the Netherlands.
- Officially, Python was made available to the public in 1991.
  The official Date of Birth for Python is: **Feb 20, 1991**.

Python is a powerful and easy-to-learn language.

Inventor: Guido van Rossum, a Dutch programmer, developed Python in 1989.

Real Scientific Applications Developed Using Python

Python is widely used in major scientific institutions for real-world, large-scale applications:

- **CERN (European Organization for Nuclear Research):**
  - Uses Python for data analysis in high-energy physics.
  - The Large Hadron Collider (LHC) software framework integrates Python for scripting and automation.
- **NASA:**
  - Uses Python in various space missions.
  - Projects like the **James Webb Space Telescope** use Python for data calibration and analysis.
- **JPL (Jet Propulsion Laboratory):**
  - Uses Python in robotic mission planning and control systems.
  - `F´ (F Prime)`, a flight software framework for CubeSats, is partially managed using Python.
- **LIGO (Laser Interferometer Gravitational-Wave Observatory):**
  - Python is used in signal processing to detect gravitational waves.
  - Libraries like `GWpy` are built for analyzing LIGO data.

Features Borrowed from Other Languages

Guido developed Python by borrowing programming features from several languages:

1. Functional Programming Features from **C**
2. Object-Oriented Programming Features from **C++**
3. Scripting Language Features from **Perl and Shell Script**
4. Modular Programming Features from **Modula-3**

What is a Scripting Language?

- A **scripting language** is a programming language designed to automate the execution of tasks. A small program meant for one-time use.
- It is usually interpreted rather than compiled.
- Scripts are often used to control software applications, perform system tasks at runtime, or manipulate data quickly.
- Examples: `Python`, `Bash`, `Perl`, `JavaScript`, `Ruby`

Hello World Program Comparison

C:

```c
#include <stdio.h>

void main()
{
    printf("Hello world");
}
```

Python:

```python
print("Hello World")
```

Sum Program Comparison

C:

```c
#include <stdio.h>

void main()
{
    int a, b;
    a = 10;
    b = 20;
    printf("The Sum: %d", (a + b));
}
```

Python:

```python
a = 10
b = 20
print("The Sum:", (a + b))
```

Python is an Interpreted Language

- Python code is executed line-by-line using a program called the **interpreter**.
- There is no separate compilation step that generates a binary.
- Python code is translated to intermediate bytecode, which is then executed by the Python Virtual Machine (PVM).

Difference Between Compiler and Interpreter:

Feature	**Interpreter (Python)**	**Compiler (C, C++)**
Execution	Line-by-line	Entire program at once
Output	No binary; runs via interpreter	Produces executable
Error Reporting	Stops at first error	Reports all errors at once
Speed	Slower	Faster after compilation

Python Execution Flow

Python Source Code (.py)

| Compilation

Bytecode (.pyc)

| Execution

Python Virtual Machine (PVM)

|

Output (Result)

- Python source code is first compiled to intermediate **bytecode**.
- Bytecode is then executed by the **Python Virtual Machine (PVM)**.

Python as an Interpreter (Interactive Mode)

Python executes code line by line at runtime.
Here is an example using the interactive terminal:

```
>>> print("How are you?")
How are you?
>>> 10 + 25
35
>>> a = 5
>>> b = 7
>>> a * b
35
```

- Python responds immediately after each line.
- No need to compile the full program.
- Great for testing and debugging small pieces of code interactively.

Differences Between Python and C++

Python	**C++**
Python is an interpreted language.	C++ is a compiled language.
Python is portable across platforms.	C++ is also portable but requires recompilation on different systems.
Python has automatic memory management with garbage collection.	C++ uses manual memory management, but supports smart pointers.
Slower execution speed due to interpretation.	Faster execution speed due to compilation.
No strict type declarations for function parameters and return types.	Requires explicit declaration of parameter and return types.

Installing Python (Windows, Mac, Linux)

1. **Windows:**
   - Visit `https://www.python.org`
   - Go to `Downloads` and click on `Python for Windows`
   - Run the installer and **check the box "Add Python to PATH"**
2. **macOS:**
   - Python 2.x is pre-installed (deprecated)
   - To install the latest version:
     - Use Homebrew: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh")`
     - Then run: `brew install python`
3. **Linux (Ubuntu/Debian-based):**
   - Open terminal and run:
   - `sudo apt update`
   - `sudo apt install python3`

Python Versions

Python has evolved over the years with major version releases introducing significant improvements and changes.

- **Python 1.0** – Introduced in **January 1994**
  - Initial public release.
  - Included core features like functions, exceptions, and modules.
- **Python 2.0** – Introduced in **October 2000**
  - Introduced list comprehensions and garbage collection.
  - Widely adopted in the 2000s.
- **Python 3.0** – Introduced in **December 2008**
  - Not backward compatible with Python 2.
  - Emphasized Unicode support and cleaner syntax.
  - Modern Python versions (3.6–3.12+) are built upon this.

Writing and Running Your First Python Program

Step 1: Write Hello World Program

```
$ gedit hello.py
# or use any editor

$ python hello.py
# Output:
Hello world
```

hello.py

```python
print("Hello world")
```

Step 2: Use Functions and main()

```
$ gedit program.py

$ python program.py
# Output:
Hello world
```

program.py

```python
def function1():
    print("Hello world")

if __name__ == "__main__":
    function1()
```

Valid Python Identifiers

Valid Python identifiers.

- `1name` ✗ (Starts with a digit)
- `name1` ✓
- `data_2025` ✓
- `my$var` ✗ (Special character '$' not allowed)
- `_temp_var_` ✓
- `class` ✗ (Python keyword)
- `import` ✗ (Python keyword)

Rules:

- Must start with a letter or underscore.
- Can contain letters, digits, and underscores.
- Cannot be a Python reserved keyword.

Python Reserved Words

There are **33 reserved keywords** in Python. These are words that have special meaning in the language and **cannot be used as identifiers**.

- `True`, `False`, `None`
- `and`, `or`, `not`, `is`
- `if`, `elif`, `else`
- `while`, `for`, `break`, `continue`, `return`, `in`, `yield`
- `try`, `except`, `finally`, `raise`, `assert`
- `import`, `from`, `as`, `class`, `def`, `pass`, `global`, `nonlocal`, `lambda`, `del`, `with`

Note: These keywords are case-sensitive (e.g., `True` is valid, but `true` is not).

Python Data Types

- Python is a **dynamically typed language** – the type is inferred at runtime.

Built-in Data Types:

- `int`
- `float`
- `complex`
- `bool`
- `str`
- `bytes`
- `bytearray`
- `range`
- `list`
- `tuple`
- `set`
- `frozenset`
- `dict`
- `None`

Built-in Functions:

- `type()` – returns the type of a variable.
- `id()` – returns the memory address of an object.
- `print()` – displays the value.

Dynamic Typing in Python

Python is dynamically typed: You do not need to declare variable types explicitly — Python assigns the type automatically based on the value.

Example Code:

```python
a = 10
b = 3.14
c = True
d = 2 + 3j
e = "Hello"
print(type(a))   # <class 'int'>
print(type(b))   # <class 'float'>
print(type(c))   # <class 'bool'>
print(type(d))   # <class 'complex'>
print(type(e))   # <class 'str'>
```

- `type()` is used to check the type of a variable.
- Python infers the type based on the assigned value.

Variable, Type, and Memory Address in Python

Introspection in Python:

- `print(a)` → prints the value → 10
- `type(a)` → returns type → `<class 'int'>`
- `id(a)` → returns memory address (unique ID) → e.g., 140736

String Indexing in Python

In Python, strings are indexed like arrays:

- Left to right: starting from `0`
- Right to left: starting from `-1`

-5	-4	-3	-2	-1
I	**I**	**S**	**E**	**R**
0	1	2	3	4

Example:

- `name = "IISER"`
- `name[0]` $\rightarrow$ `'I'`, `name[-1]` $\rightarrow$ `'R'`

String Slicing in Python

Python Code

```python
name = "PYTHON"

# Slicing with positive index
print(name[1:4])    # YTH

# Slicing with negative index
print(name[-5:-2]) # YTH

# Full reverse using negative step
print(name[::-1])  # NOHTYP
```

String: "PYTHON"

-6	-5	-4	-3	-2	-1
P	Y	T	H	O	N
0	1	2	3	4	5

Understanding `name[::-1]`

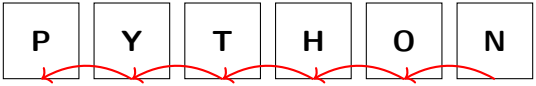
Syntax: `name[start : stop : step]`

- `start`, `stop` are omitted → entire string is selected
- `step = -1` → reverses the string

Example:

```python
name = "PYTHON"
print(name[::-1])  # Output: NOHTYP
```

Diagram – Traversing in Reverse Order:



`name[::-1]` → `"NOHTYP"`

String Multiplication in Python

- Python strings can be multiplied by an integer.
- The result is multiple copies of the string concatenated together.

```
>>> "hello" * 3
'hellohellohello'

>>> print(10 * "yo ")
yo yo yo yo yo yo yo yo yo yo

>>> print(2 * 3 * "4")
444444
```

Note:

- Multiplication works as: `string * integer`
- You can also multiply expressions like `2 * 3 * "4"` → 6 copies of "4"

String Concatenation in Python

- You cannot concatenate a string with an integer directly.
- It will raise a `TypeError`.
- **Workarounds:**
  - Use `str(value)` to convert a number to a string.
  - Use comma `print(expr1, expr2)` to print multiple values.

Examples:

```
>>> x = 4
>>> print("Thou shalt not count to " + x + ".")
TypeError: cannot concatenate 'str' and 'int' objects

>>> print("Thou shalt not count to " + str(x) + ".")
Thou shalt not count to 4.

>>> print(x + 1, "is out of the question.")
5 is out of the question.
```

Type Casting in Python (1/2)

- Type casting means converting one data type into another.
- Python provides the following inbuilt functions for type conversion:
  1. `int()`
  2. `float()`
  3. `complex()`
  4. `bool()`
  5. `str()`

Type Casting in Python (2/2)

Note:

1. We can convert from any type to another except from complex to int or float.
2. When converting from string, the string must represent a valid literal in base-10.

int() Conversion

- Converts values to integers (base-10 only).
- Cannot convert complex numbers.

```
int(123.987)      # 123
int(True)         # 1
int(False)        # 0
int("10")         # 10
int("10.5")       # Error
int(10+5j)        # Error
int("0B1111")     # Error
```

float() Conversion

- Converts values to float (base-10).
- Cannot convert complex numbers.

```
float(10)          # 10.0
float(True)        # 1.0
float(False)       # 0.0
float("10.5")      # 10.5
float("ten")       # Error
float(10+5j)       # Error
float("0B1111")    # Error
```

complex() Conversion (1/2)

- `complex(x)` creates a complex number with real part `x`, imaginary part 0.

```
complex(10)        # 10+0j
complex(10.5)      # 10.5+0j
complex(True)      # 1+0j
complex(False)     # 0j
complex("10")      # 10+0j
complex("10.5")    # 10.5+0j
complex("ten")     # Error
```

complex() Conversion (2/2)

- `complex(x, y)` creates complex number with real part `x`, imag part `y`.

```
complex(10, -2)          # 10-2j
complex(True, False)     # 1+0j
```

bool() Conversion

- Converts values to `True` or `False`.
- Zero-like values (0, 0.0, 0j, "", etc.) become `False`, others become `True`.

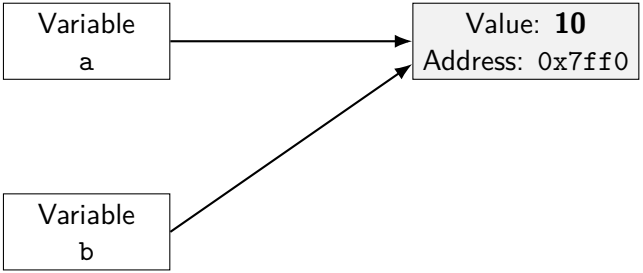
```
bool(0)              # False
bool(1)              # True
bool(10)             # True
bool(10.5)           # True
bool(0+0j)           # False
bool(10-2j)          # True
bool(0+1.5j)         # True
bool("True")         # True
bool("False")        # True
bool("")             # False
```

Mutable vs Immutable in Python (1/2)

- **Immutable types:** Their content cannot be changed after creation.
- Example types: `int`, `float`, `bool`, `str`, `tuple`.
- If two variables store the same immutable value, Python may assign them the same memory address to optimize performance.

```
>>> a = 10
>>> b = 10
>>> id(a)
1572353952
>>> id(b)
1572353952
>>> a is b
True
```

Shared Memory for Immutable Types



In Python, immutable values like integers are shared in memory.

`a = 10`, `b = 10` $\Rightarrow$ Both point to the same object.

Mutable vs Immutable Objects

- In Python, **immutable objects** cannot be changed after creation: e.g., `int`, `float`, `bool`, `str`, `tuple`.
- **Mutable objects** can be changed after creation: e.g., `list`, `set`, `dict`.

Examples (Observed Using `id()` and `is`):

Immutable Types

- `a = 10`, `b = 10` ⇒ `a is b`: True
- `a = 'iiser'`, `b = 'iiser'` ⇒ `a is b`: True
- `a = 10 + 5j`, `b = 10 + 5j` ⇒ `a is b`: False
- `a = True`, `b = True` ⇒ `a is b`: True

Mutable Types

- `a = [10, 20]`, `b = [10, 20]` ⇒ `a is b`: False
- `a = [10, 20]`, `b = a` ⇒ `a is b`: True
- Modifying `b` (e.g., `b[0] = 99`) affects `a`
- `id(a) == id(b)` if same reference

Arithmetic Operators in Python (1/2)

- + : Addition
- - : Subtraction
- * : Multiplication
- / : Division (returns float)
- // : Floor Division (returns int)

```python
a = 15
b = 4

print(a + b)     # 19
print(a - b)     # 11
print(a * b)     # 60
print(a / b)     # 3.75
print(a // b)    # 3
```

Arithmetic Operators in Python (2/2)

- % : Modulus (remainder)
- ** : Exponentiation

```python
a = 15
b = 4

print(a % b)     # 3
print(b ** 2)    # 16
print(2 ** 5)    # 32
```

Thank You!

Questions are welcome.